

Smart Invoice & Vendor Management System

A complete phase-by-phase build guide using Qt 6, C++, and SQLite

Tech Stack	Qt 6 Widgets, C++17, SQLite, CMake
Pattern	Layered architecture (UI / Service / Model / Core)
Build System	CMake with Qt Creator
Level	Intermediate to Advanced
Phases	11 phases from setup to final polish

How to use this guide: Read each phase top to bottom. Type out the code yourself, do not copy paste. At the end of each phase there is a checkpoint. Run the app and confirm everything works before moving to the next phase. If something breaks, read the error and fix it before continuing. Broken code compounds fast.

Table of Contents

Section	Topic
Preface	What you need to know before starting
Phase 1	Project setup, Qt install, CMake, database connection
Phase 2	Authentication: signup, login, password hashing
Phase 3	Dashboard and navigation
Phase 4	Vendor CRUD and search
Phase 5	Invoice UI and item table (QAbstractTableModel)
Phase 6	GST calculation engine
Phase 7	Auto fill vendor with QCompleter
Phase 8	Invoice history with QSortFilterProxyModel
Phase 9	PDF export with QPainter and QPdfWriter
Phase 10	Settings, persistence, dark mode
Phase 11	Optimization, refactoring, packaging
Appendix A	Common Qt mistakes and how to debug them
Appendix B	Final project checklist

Preface: What You Need Before Starting

You already know C++. Good. What you do not know yet is the Qt framework, its meta object system, its signal slot mechanism, its model view architecture, and how it talks to SQL. This guide assumes you can read and write standard C++17. Anything Qt specific gets explained.

The five Qt concepts you must understand first

1. QObject and the meta object system. Qt extends C++ with reflection. Any class that inherits from QObject and uses the Q_OBJECT macro gets processed by a tool called the Meta Object Compiler (MOC) before regular C++ compilation. MOC generates extra code that powers signals, slots, properties, and introspection. You never run MOC by hand. CMake does it through the AUTOMOC flag.

2. Signals and slots. This is Qt's event system. A signal is emitted when something happens (button clicked, text changed). A slot is a function that runs in response. You connect them with `QObject::connect(sender, &Class::signal, receiver, &Class::slot)`. This replaces the callback pattern you would use in plain C++.

3. Parent child ownership. Qt manages memory through a parent tree. When you pass a parent pointer to a widget constructor, the parent takes ownership and deletes the child when the parent dies. This means you create widgets with new and almost never call delete on them yourself.

4. Model view framework. Qt separates data (model) from display (view). A QTableView shows data from a QAbstractTableModel. You implement the model, Qt handles the rendering. This is the cleanest way to build editable tables, which you will need for invoice items.

5. QSqlDatabase and QSqlQuery. Qt has its own SQL module. You open a connection, then run queries through QSqlQuery. Always use prepared statements with bindValue. Never concatenate strings into SQL.

Tools you need installed

- Qt 6.6 or newer (the open source installer from qt.io)
- A C++17 compiler (MinGW on Windows, GCC on Linux, Clang on macOS)
- Qt Creator IDE (ships with the Qt installer)
- CMake 3.16 or newer (also ships with the Qt installer)
- A SQLite browser like DB Browser for SQLite, optional but useful for inspecting the database

A small decision you must make now

Your project plan uses the word 'vendor' but the GST flow (CGST, SGST, IGST collected on sales) is a sales invoicing flow. In Indian accounting, a vendor is who you buy from, and a customer is who you sell to. Decide which one your app does. This guide assumes sales invoicing, so 'vendor' in our schema actually means 'customer you bill'. If you want a pure purchase tracker, the GST direction flips (input tax credit instead of output tax). For a portfolio project, sales invoicing is the standard choice and what we build here.

Phase 1: Project Setup and Database Connection

Goal of this phase: install Qt, create an empty Qt Widgets project that compiles, add the SQL module, wire up a singleton Database class, and confirm the SQLite file gets created on first run.

Step 1.1: Install Qt 6

Download the Qt Online Installer from qt.io/download-qt-installer. During installation choose: Qt 6.7 or newer (LTS preferred), a compiler kit (MinGW on Windows is simplest), Qt Creator, and CMake. On Ubuntu you can use apt:

```
sudo apt install qt6-base-dev qt6-tools-dev qt6-tools-dev-tools \
    libqt6sql6-sqlite cmake build-essential
```

After installation, open Qt Creator once and let it auto detect kits. A kit bundles a Qt version with a compiler and a debugger. If no kit shows up, go to Edit > Preferences > Kits and add one manually.

Step 1.2: Create the project

Open Qt Creator, click 'Create Project' > 'Application (Qt)' > 'Qt Widgets Application'. Set the name to **InvoiceApp**, build system **CMake**, base class **QMainWindow**, and uncheck 'Generate form file'. We write UI in code so you learn what is happening under the hood.

Step 1.3: Folder structure

Inside the project root, create these folders (use your OS file manager or the terminal):

```
InvoiceApp/
■■■ main.cpp
■■■ CMakeLists.txt
■■■ core/
■■■ models/
■■■ services/
■■■ ui/
■   ■■■ components/
■■■ resources/
■   ■■■ icons/
■   ■■■ styles/
■   ■■■ logo/
■■■ database/
■■■ exports/
    ■■■ invoices/
```

Step 1.4: Update CMakeLists.txt

Qt Creator generates a default CMakeLists. Replace it with this version. The important additions are **Sql** in find_package, AUTOMOC and AUTORCC flags, and the source file list:

```
cmake_minimum_required(VERSION 3.16)
project(InvoiceApp VERSION 0.1 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_AUTOMOC ON)
set(CMAKE_AUTORCC ON)
set(CMAKE_AUTOUIC ON)

find_package(Qt6 6.5 REQUIRED COMPONENTS Widgets Sql PrintSupport)
```

```

set(SOURCES
    main.cpp
    core/database.cpp
    core/sessionmanager.cpp
)

set(HEADERS
    core/database.h
    core/sessionmanager.h
)

qt_add_executable(InvoiceApp ${SOURCES} ${HEADERS})

target_link_libraries(InvoiceApp PRIVATE
    Qt6::Widgets
    Qt6::Sql
    Qt6::PrintSupport
)

target_include_directories(InvoiceApp PRIVATE ${CMAKE_SOURCE_DIR})

```

Note: You will add more files to the SOURCES and HEADERS lists as the project grows. Forgetting to add a new .cpp file here is the most common 'undefined reference' error in Qt projects. Every time you create a new class, update CMakeLists immediately.

Step 1.5: Write the Database singleton

The Database class manages one SQLite connection for the whole app. We use the Meyers singleton pattern (a static local variable inside a function), which is thread safe since C++11.

core/database.h

```

#ifndef DATABASE_H
#define DATABASE_H

#include <QSqlDatabase>
#include <QString>

class Database {
public:
    // Singleton access point. Same instance for the whole app.
    static Database& instance();

    // Open the SQLite file. Call once at app startup.
    bool connect(const QString& filePath);

    // Returns the live QSqlDatabase handle.
    QSqlDatabase& db();

    // Create all tables if they do not exist.
    bool initSchema();

private:
    Database() = default; // private constructor
    Database(const Database&) = delete; // no copy
    Database& operator=(const Database&) = delete;

    QSqlDatabase m_db;
};

#endif // DATABASE_H

```

core/database.cpp

```
#include "database.h"
#include <QSqlError>
#include <QSqlQuery>
#include <QFileInfo>
#include <QDir>
#include <QDebug>

Database& Database::instance() {
    static Database d;
    return d;
}

bool Database::connect(const QString& filePath) {
    // Make sure the parent folder exists before SQLite tries to write.
    QFileInfo info(filePath);
    QDir().mkpath(info.absolutePath());

    m_db = QSqlDatabase::addDatabase("SQLITE");
    m_db.setDatabaseName(filePath);

    if (!m_db.open()) {
        qWarning() << "DB open failed:" << m_db.lastError().text();
        return false;
    }

    // Turn on foreign keys (off by default in SQLite).
    QSqlQuery pragma(m_db);
    pragma.exec("PRAGMA foreign_keys = ON;");
    return true;
}

QSqlDatabase& Database::db() {
    return m_db;
}

bool Database::initSchema() {
    QSqlQuery q(m_db);

    const QStringList tables = {
        "CREATE TABLE IF NOT EXISTS users ("
        "    id INTEGER PRIMARY KEY AUTOINCREMENT,"
        "    username TEXT UNIQUE NOT NULL,"
        "    password_hash TEXT NOT NULL,"
        "    salt TEXT NOT NULL,"
        "    created_at TEXT NOT NULL)",

        "CREATE TABLE IF NOT EXISTS vendors ("
        "    id INTEGER PRIMARY KEY AUTOINCREMENT,"
        "    name TEXT NOT NULL,"
        "    gst TEXT,"
        "    phone TEXT,"
        "    email TEXT,"
        "    address TEXT,"
        "    tax_details TEXT,"
        "    created_at TEXT NOT NULL)",

        "CREATE TABLE IF NOT EXISTS invoices ("
        "    id INTEGER PRIMARY KEY AUTOINCREMENT,"
        "    invoice_no TEXT UNIQUE NOT NULL,"
        "    vendor_id INTEGER NOT NULL,"
        "    invoice_date TEXT NOT NULL,"
        "    due_date TEXT,"
        "    subtotal REAL NOT NULL DEFAULT 0,"
```

```

" gst_total REAL NOT NULL DEFAULT 0,"
" grand_total REAL NOT NULL DEFAULT 0,"
" status TEXT NOT NULL DEFAULT 'Pending',"
" notes TEXT,"
" created_at TEXT NOT NULL,"
" FOREIGN KEY(vendor_id) REFERENCES vendors(id))",

"CREATE TABLE IF NOT EXISTS invoice_items ("
" id INTEGER PRIMARY KEY AUTOINCREMENT,"
" invoice_id INTEGER NOT NULL,"
" item_name TEXT NOT NULL,"
" quantity INTEGER NOT NULL DEFAULT 1,"
" price REAL NOT NULL DEFAULT 0,"
" gst_percent REAL NOT NULL DEFAULT 0,"
" cgst REAL NOT NULL DEFAULT 0,"
" sgst REAL NOT NULL DEFAULT 0,"
" igst REAL NOT NULL DEFAULT 0,"
" total REAL NOT NULL DEFAULT 0,"
" FOREIGN KEY(invoice_id) REFERENCES invoices(id) ON DELETE CASCADE)",

"CREATE TABLE IF NOT EXISTS company_settings ("
" id INTEGER PRIMARY KEY CHECK (id = 1),"
" company_name TEXT, company_gst TEXT,"
" address TEXT, phone TEXT, email TEXT,"
" logo_path TEXT)"
};

for (const QString& sql : tables) {
    if (!q.exec(sql)) {
        qWarning() << "Schema error:" << q.lastError().text();
        return false;
    }
}
return true;
}

```

Step 1.6: Hook it up in main.cpp

```
#include <QApplication>
#include <QMessageBox>
#include <QStandardPaths>
#include <QDir>
#include "core/database.h"

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);

    // Store the db inside the user app data folder so it survives reinstalls.
    const QString dataDir =
        QStandardPaths::writableLocation(QStandardPaths::AppDataLocation);
    const QString dbPath = dataDir + "/invoice.db";

    if (!Database::instance().connect(dbPath)) {
        QMessageBox::critical(nullptr, "Fatal", "Cannot open database.");
        return 1;
    }
    if (!Database::instance().initSchema()) {
        QMessageBox::critical(nullptr, "Fatal", "Cannot init schema.");
        return 1;
    }

    QMessageBox::information(nullptr, "OK",
        "Database is ready at:\n" + dbPath);
    return app.exec();
}
```

Checkpoint for Phase 1

Build and run. You should see a message box with the path to your invoice.db file. Open that file in DB Browser for SQLite. You should see five tables: users, vendors, invoices, invoice_items, company_settings. If any of this fails, do not continue. Common causes: Qt6Sql.dll not deployed, Sql module missing in CMakeLists, or the data directory is not writable.

Phase 2: Authentication System

Goal: build the login and signup windows, store user passwords hashed with a per user salt, and add a session manager that tracks who is logged in.

Step 2.1: Models and SessionManager

models/user.h

```
#ifndef USER_H
#define USER_H

#include <QString>

struct User {
    int id = -1;
    QString username;
    QString passwordHash;
    QString salt;
    QString createdAt;
};

#endif
```

core/sessionmanager.h

```
#ifndef SESSIONMANAGER_H
#define SESSIONMANAGER_H

#include "models/user.h"

class SessionManager {
public:
    static SessionManager& instance();
    void setUser(const User& u);
    void clear();
    bool isLoggedIn() const;
    const User& currentUser() const;

private:
    SessionManager() = default;
    User m_user;
    bool m_loggedIn = false;
};

#endif
```

core/sessionmanager.cpp

```
#include "sessionmanager.h"

SessionManager& SessionManager::instance() {
    static SessionManager s;
    return s;
}

void SessionManager::setUser(const User& u) { m_user = u; m_loggedIn = true; }
void SessionManager::clear() { m_user = User(); m_loggedIn = false; }
bool SessionManager::isLoggedIn() const { return m_loggedIn; }
const User& SessionManager::currentUser() const { return m_user; }
```

Step 2.2: AuthService with salted SHA256

Plain SHA256 of a password is weak because two users with the same password share the same hash, and rainbow tables crack them in seconds. We add a random per user salt. For real production you would use Argon2 or bcrypt, but for a desktop portfolio project salted SHA256 is acceptable and easy to implement with QCryptographicHash.

services/authservice.h

```
#ifndef AUTHSERVICE_H
#define AUTHSERVICE_H

#include <QString>
#include "models/user.h"

class AuthService {
public:
    // Returns true and fills outUser on success.
    bool signup(const QString& username, const QString& password,
               QString* errorOut, User* outUser = nullptr);

    bool login(const QString& username, const QString& password,
              QString* errorOut, User* outUser);

private:
    static QString makeSalt();
    static QString hashWithSalt(const QString& password, const QString& salt);
};

#endif
```

services/authservice.cpp

```
#include "authservice.h"
#include "core/database.h"
#include <QCryptographicHash>
#include <QSqlQuery>
#include <QSqlError>
#include <QDateTime>
#include <QRandomGenerator>

QString AuthService::makeSalt() {
    // 16 random bytes encoded as hex -> 32 char string.
    QByteArray bytes(16, 0);
    for (int i = 0; i < bytes.size(); ++i)
        bytes[i] = static_cast<char>(QRandomGenerator::global()->bounded(256));
    return QString(bytes.toHex());
}

QString AuthService::hashWithSalt(const QString& password, const QString& salt) {
    QByteArray combined = (salt + password).toUtf8();
    QByteArray hash = QCryptographicHash::hash(combined,
                                                QCryptographicHash::Sha256);
    return QString(hash.toHex());
}

bool AuthService::signup(const QString& username, const QString& password,
                        QString* err, User* outUser) {
    if (username.size() < 3) { *err = "Username too short"; return false; }
    if (password.size() < 6) { *err = "Password too short"; return false; }

    QSqlQuery q(Database::instance().db());
    q.prepare("SELECT id FROM users WHERE username = :u");
    q.bindValue(":u", username);
    q.exec();
    if (q.next()) { *err = "Username already exists"; return false; }

    QString salt = makeSalt();
    QString hash = hashWithSalt(password, salt);
    QString now = QDateTime::currentDateTime().toString(Qt::ISODate);

    q.prepare("INSERT INTO users(username, password_hash, salt, created_at) "
              "VALUES(:u, :h, :s, :t)");
    q.bindValue(":u", username);
    q.bindValue(":h", hash);
    q.bindValue(":s", salt);
    q.bindValue(":t", now);
    if (!q.exec()) { *err = q.lastError().text(); return false; }

    if (outUser) {
        outUser->id = q.lastInsertId().toInt();
        outUser->username = username;
        outUser->passwordHash = hash;
        outUser->salt = salt;
        outUser->createdAt = now;
    }
    return true;
}

bool AuthService::login(const QString& username, const QString& password,
                      QString* err, User* outUser) {
    QSqlQuery q(Database::instance().db());
    q.prepare("SELECT id, password_hash, salt, created_at FROM users "
              "WHERE username = :u");
    q.bindValue(":u", username);
    q.exec();
}
```

```

        if (!q.next()) { *err = "User not found"; return false; }

        QString storedHash = q.value(1).toString();
        QString storedSalt = q.value(2).toString();
        if (hashWithSalt(password, storedSalt) != storedHash) {
            *err = "Wrong password";
            return false;
        }

        outUser->id            = q.value(0).toInt();
        outUser->username       = username;
        outUser->passwordHash   = storedHash;
        outUser->salt           = storedSalt;
        outUser->createdAt      = q.value(3).toString();
        return true;
    }
}

```

Step 2.3: LoginWindow UI

Build the UI in code so you see every widget being created. Once you understand the layout system, you can switch to Qt Designer .ui files later if you want.

ui/loginwindow.h

```

#ifndef LOGINWINDOW_H
#define LOGINWINDOW_H

#include <QWidget>
class QLineEdit;
class QPushButton;
class QLabel;

class LoginWindow : public QWidget {
    Q_OBJECT
public:
    explicit LoginWindow(QWidget* parent = nullptr);

signals:
    void loggedIn();           // fired when login succeeds
    void openSignup();         // user wants to switch to signup

private slots:
    void onLoginClicked();

private:
    QLineEdit*    m_username;
    QLineEdit*    m_password;
    QPushButton*  m_loginBtn;
    QPushButton*  m_signupBtn;
    QLabel*       m_status;
};

#endif

```

ui/loginwindow.cpp

```
#include "loginwindow.h"
#include "services/authservice.h"
#include "core/sessionmanager.h"

#include <QLineEdit>
#include <QPushButton>
#include <QLabel>
#include <QVBoxLayout>
#include <QFormLayout>

LoginWindow::LoginWindow(QWidget* parent) : QWidget(parent) {
    setWindowTitle("Login - Invoice App");
    setMinimumWidth(360);

    m_username = new QLineEdit(this);
    m_password = new QLineEdit(this);
    m_password->setEchoMode(QLineEdit::Password);

    m_loginBtn = new QPushButton("Login", this);
    m_signupBtn = new QPushButton("Create account", this);
    m_status = new QLabel(this);
    m_status->setStyleSheet("color: #b00020;");

    auto* form = new QFormLayout;
    form->addRow("Username:", m_username);
    form->addRow("Password:", m_password);

    auto* root = new QVBoxLayout(this);
    root->addLayout(form);
    root->addWidget(m_loginBtn);
    root->addWidget(m_signupBtn);
    root->addWidget(m_status);

    connect(m_loginBtn, &QPushButton::clicked,
            this, &LoginWindow::onLoginClicked);
    connect(m_signupBtn, &QPushButton::clicked,
            this, &LoginWindow::openSignup);
}

void LoginWindow::onLoginClicked() {
    AuthService auth;
    QString err;
    User u;
    if (auth.login(m_username->text().trimmed(),
                  m_password->text(), &err, &u)) {
        SessionManager::instance().setUser(u);
        emit loggedIn();
    } else {
        m_status->setText(err);
    }
}
```

Step 2.4: SignupWindow (short version)

SignupWindow is structurally identical to LoginWindow with three fields (username, password, confirm) and a call to AuthService::signup. Build it yourself as practice. The pattern is the same: create widgets, lay them out, connect the button, call the service, react to the result.

Step 2.5: Wire everything in main.cpp

```
#include <QApplication>
#include "core/database.h"
#include "ui/loginwindow.h"
#include "ui/signupwindow.h"
#include "ui/dashboardwindow.h" // built in Phase 3, comment out for now

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    Database::instance().connect("invoice.db");
    Database::instance().initSchema();

    auto* login = new LoginWindow;
    auto* signup = new SignupWindow;

    QObject::connect(login, &LoginWindow::openSignup, [=]{
        login->hide(); signup->show();
    });
    QObject::connect(signup, &SignupWindow::backToLogin, [=]{
        signup->hide(); login->show();
    });
    QObject::connect(login, &LoginWindow::loggedIn, [=]{
        login->hide();
        // auto* dash = new DashboardWindow; dash->show(); // Phase 3
    });

    login->show();
    return app.exec();
}
```

Checkpoint for Phase 2

- App opens to the login screen.
- Clicking 'Create account' shows the signup window.
- Creating a user inserts a row in the users table with a non-empty salt and hash.
- Logging in with correct credentials emits loggedIn. Wrong password shows an error.
- Inspect the users table in DB Browser to confirm salts are random per user.

Phase 3: Dashboard and Navigation

The dashboard is the central hub. From here the user opens vendors, creates invoices, sees history, and changes settings. We use a `QStackedWidget` so the same window swaps between pages instead of opening separate windows. This is the modern desktop UX pattern.

Step 3.1: DashboardWindow header

ui/dashboardwindow.h

```
#ifndef DASHBOARDWINDOW_H
#define DASHBOARDWINDOW_H

#include <QMainWindow>
class QStackedWidget;
class QListWidget;
class QLabel;

class DashboardWindow : public QMainWindow {
    Q_OBJECT
public:
    explicit DashboardWindow(QWidget* parent = nullptr);

signals:
    void loggedOut();

private slots:
    void onNavChanged(int row);
    void onLogoutClicked();

private:
    void buildUI();
    void refreshStats();

    QListWidget* m_nav;
    QStackedWidget* m_pages;
    QLabel* m_statTotalInvoices;
    QLabel* m_statTotalRevenue;
    QLabel* m_statPending;
};

#endif
```

ui/dashboardwindow.cpp

```
#include "dashboardwindow.h"
#include "vendorwindow.h"
#include "invoicewindow.h"
#include "historywindow.h"
#include "settingswindow.h"
#include "core/sessionmanager.h"
#include "core/database.h"

#include <QStackedWidget>
#include <QListWidget>
#include <QLabel>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QPushButton>
#include <QSqlQuery>

DashboardWindow::DashboardWindow(QWidget* parent) : QMainWindow(parent) {
    setWindowTitle("Invoice App - Dashboard");
    resize(1100, 700);
    buildUI();
    refreshStats();
}

void DashboardWindow::buildUI() {
    // ---- Sidebar nav ----
    m_nav = new QListWidget;
    m_nav->addItems({"Home", "Vendors", "New Invoice", "History", "Settings"});
    m_nav->setFixedWidth(190);
    m_nav->setStyleSheet("QListWidget { background:#0b3d91; color:white; }"
        "QListWidget::item { padding:14px; }"
        "QListWidget::item:selected { background:#1a4fb0; }");

    // ---- Pages ----
    m_pages = new QStackedWidget;

    // Home page with stats
    auto* home = new QWidget;
    m_statTotalInvoices = new QLabel("0");
    m_statTotalRevenue = new QLabel("0.00");
    m_statPending = new QLabel("0");

    auto statsRow = new QHBoxLayout;
    auto makeCard = [](const QString& title, QLabel* value) {
        auto* card = new QWidget;
        card->setStyleSheet("background:#f4f6fa; border-radius:8px;");
        auto* v = new QVBoxLayout(card);
        auto* t = new QLabel(title);
        t->setStyleSheet("color:#666;");
        value->setStyleSheet("font-size:28px; font-weight:bold; color:#0b3d91;");
        v->addWidget(t); v->addWidget(value);
        return card;
    };
    statsRow->addWidget(makeCard("Total Invoices", m_statTotalInvoices));
    statsRow->addWidget(makeCard("Total Revenue", m_statTotalRevenue));
    statsRow->addWidget(makeCard("Pending", m_statPending));

    auto* welcome = new QLabel(
        QString("<h2>Welcome, %1</h2>")
        .arg(SessionManager::instance().currentUser().username));

    auto* logoutBtn = new QPushButton("Log out");
    connect(logoutBtn, &QPushButton::clicked,
        this, &DashboardWindow::onLogoutClicked);
}
```



```

auto* hv = new QVBoxLayout(home);
hv->addWidget(welcome);
hv->addLayout(statsRow);
hv->addStretch();
hv->addWidget(logoutBtn);

m_pages->addWidget(home); // index 0
m_pages->addWidget(new VendorWindow); // index 1
m_pages->addWidget(new InvoiceWindow); // index 2
m_pages->addWidget(new HistoryWindow); // index 3
m_pages->addWidget(new SettingsWindow); // index 4

connect(m_nav, &QListWidget::currentRowChanged,
        this, &DashboardWindow::onNavChanged);
m_nav->setCurrentRow(0);

// ---- Compose ----
auto* central = new QWidget;
auto* h = new QHBoxLayout(central);
h->setContentsMargins(0, 0, 0, 0);
h->setSpacing(0);
h->addWidget(m_nav);
h->addWidget(m_pages, 1);
setCentralWidget(central);
}

void DashboardWindow::onNavChanged(int row) {
    m_pages->setCurrentIndex(row);
    if (row == 0) refreshStats();
}

void DashboardWindow::onLogoutClicked() {
    SessionManager::instance().clear();
    emit loggedOut();
    close();
}

void DashboardWindow::refreshStats() {
    QSqlQuery q(Database::instance().db());
    q.exec("SELECT COUNT(*), COALESCE(SUM(grand_total),0) FROM invoices");
    if (q.next()) {
        m_statTotalInvoices->setText(QString::number(q.value(0).toInt()));
        m_statTotalRevenue->setText(
            QString::number(q.value(1).toDouble(), 'f', 2));
    }
    q.exec("SELECT COUNT(*) FROM invoices WHERE status='Pending'");
    if (q.next()) m_statPending->setText(QString::number(q.value(0).toInt()));
}

```

Checkpoint for Phase 3

The dashboard shows a sidebar with five items, a welcome label, and three stat cards (all zero until you add invoices). Clicking sidebar items switches the right hand panel. Logout returns to login. VendorWindow, InvoiceWindow, HistoryWindow, SettingsWindow can be empty QWidget placeholders for now.

Phase 4: Vendor CRUD and Search

Goal: list vendors in a QTableView, add new ones, edit, delete, and search by name or GST.

Step 4.1: VendorService

models/vendor.h

```
#ifndef VENDOR_H
#define VENDOR_H

#include <QString>

struct Vendor {
    int    id = -1;
    QString name;
    QString gst;
    QString phone;
    QString email;
    QString address;
    QString taxDetails;
    QString createdAt;
};

#endif
```

services/vendorservice.h

```
#ifndef VENDORSERVICE_H
#define VENDORSERVICE_H

#include <QList>
#include "models/vendor.h"

class VendorService {
public:
    QList<Vendor> all(const QString& search = QString());
    bool insert(Vendor* v, QString* err);
    bool update(const Vendor& v, QString* err);
    bool remove(int id, QString* err);
    bool findById(int id, Vendor* out);
};

#endif
```

services/vendorservice.cpp

```
#include "vendorservice.h"
#include "core/database.h"
#include < QSqlQuery >
#include < QSqlError >
#include < QDateTime >

QList<Vendor> VendorService::all(const QString& search) {
    QList<Vendor> list;
    QSqlQuery q(Database::instance().db());

    if (search.isEmpty()) {
        q.prepare("SELECT id,name,gst,phone,email,address,tax_details,created_at "
                  "FROM vendors ORDER BY name");
    } else {
        q.prepare("SELECT id,name,gst,phone,email,address,tax_details,created_at "
                  "FROM vendors "
                  "WHERE name LIKE :s OR gst LIKE :s OR phone LIKE :s "
                  "ORDER BY name");
        q.bindValue(":s", "%" + search + "%");
    }
    q.exec();
    while (q.next()) {
        Vendor v;
        v.id = q.value(0).toInt();
        v.name = q.value(1).toString();
        v.gst = q.value(2).toString();
        v.phone = q.value(3).toString();
        v.email = q.value(4).toString();
        v.address = q.value(5).toString();
        v.taxDetails = q.value(6).toString();
        v.createdAt = q.value(7).toString();
        list.append(v);
    }
    return list;
}

bool VendorService::insert(Vendor* v, QString* err) {
    if (v->name.trimmed().isEmpty()) { *err = "Name required"; return false; }

    QSqlQuery q(Database::instance().db());
    q.prepare("INSERT INTO vendors(name,gst,phone,email,address,tax_details,created_at) "
              "VALUES(:n,:g,:p,:e,:a,:t,:c)");
    q.bindValue(":n", v->name);
    q.bindValue(":g", v->gst);
    q.bindValue(":p", v->phone);
    q.bindValue(":e", v->email);
    q.bindValue(":a", v->address);
    q.bindValue(":t", v->taxDetails);
    q.bindValue(":c", QDateTime::currentDateTime().toString(Qt::ISODate));
    if (!q.exec()) { *err = q.lastError().text(); return false; }
    v->id = q.lastInsertId().toInt();
    return true;
}

bool VendorService::update(const Vendor& v, QString* err) {
    QSqlQuery q(Database::instance().db());
    q.prepare("UPDATE vendors SET name=:n, gst=:g, phone=:p, email=:e, "
              "address=:a, tax_details=:t WHERE id=:id");
    q.bindValue(":n", v.name);
    q.bindValue(":g", v.gst);
    q.bindValue(":p", v.phone);
    q.bindValue(":e", v.email);
    q.bindValue(":a", v.address);
}
```

```

        q.bindValue(":t", v.taxDetails);
        q.bindValue(":id", v.id);
        if (!q.exec()) { *err = q.lastError().text(); return false; }
        return true;
    }

    bool VendorService::remove(int id, QString* err) {
        QSqlQuery q(Database::instance().db());
        q.prepare("DELETE FROM vendors WHERE id=:id");
        q.bindValue(":id", id);
        if (!q.exec()) { *err = q.lastError().text(); return false; }
        return true;
    }
}

```

Step 4.2: VendorWindow with QTableView

We use QStandardItemModel for simplicity. Once you are comfortable, you can graduate to a custom QAbstractTableModel like we do for invoice items in Phase 5. Both approaches work.

ui/vendorwindow.cpp (key parts)

```
#include "vendorwindow.h"
#include "services/vendorservice.h"

#include <QTableView>
#include <QStandardItemModel>
#include <QPushButton>
#include <QLineEdit>
#include <QHBoxLayout>
#include <QVBoxLayout>
#include <QMessageBox>
#include <QHeaderView>

VendorWindow::VendorWindow(QWidget* parent) : QWidget(parent) {
    m_search = new QLineEdit;
    m_search->setPlaceholderText("Search by name, GST or phone");

    m_table = new QTableView;
    m_model = new QStandardItemModel(this);
    m_table->setModel(m_model);
    m_table->setSelectionBehavior(QAbstractItemView::SelectRows);
    m_table->setSelectionMode(QAbstractItemView::SingleSelection);
    m_table->horizontalHeader()->setStretchLastSection(true);

    auto* addBtn = new QPushButton("Add");
    auto* editBtn = new QPushButton("Edit");
    auto* delBtn = new QPushButton("Delete");

    auto* topRow = new QHBoxLayout;
    topRow->addWidget(m_search, 1);
    topRow->addWidget(addBtn);
    topRow->addWidget(editBtn);
    topRow->addWidget(delBtn);

    auto* v = new QVBoxLayout(this);
    v->addLayout(topRow);
    v->addWidget(m_table);

    connect(m_search, &QLineEdit::textChanged,
            this, &VendorWindow::reload);
    connect(addBtn, &QPushButton::clicked,
            this, &VendorWindow::onAdd);
    connect(editBtn, &QPushButton::clicked,
            this, &VendorWindow::onEdit);
    connect(delBtn, &QPushButton::clicked,
            this, &VendorWindow::onDelete);

    reload();
}

void VendorWindow::reload() {
    m_model->clear();
    m_model->setHorizontalHeaderLabels(
        {"ID", "Name", "GST", "Phone", "Email", "Address"});

    VendorService svc;
    for (const Vendor& v : svc.all(m_search->text())) {
        QList<QStandardItem*> row;
        row << new QStandardItem(QString::number(v.id))
            << new QStandardItem(v.name)
            << new QStandardItem(v.gst)
            << new QStandardItem(v.phone)
            << new QStandardItem(v.email)
            << new QStandardItem(v.address);
    }
}
```

```
        for (auto* item : row) item->setEditable(false);  
        m_model->appendRow(row);  
    }  
}
```

onAdd, onEdit, and onDelete open a small dialog (QDialog subclass) with line edits for the vendor fields and call VendorService. The pattern is the same each time: build the form, accept the dialog, call the service, reload the table. Implement this yourself, it is good practice.

Checkpoint for Phase 4

- Vendors page lists all rows from the vendors table.
- Add a vendor, the row appears in the table.
- Edit a vendor, changes persist after restarting the app.
- Delete a vendor, it disappears. (Note: if invoices reference a vendor, the foreign key will block deletion. Handle this error and tell the user.)
- Search updates the list as you type.

Phase 5: Invoice UI and Custom Item Model

This is the biggest phase. The invoice window has top fields (invoice number, date, vendor), a table of line items, and a totals section. The table uses a custom `QAbstractTableModel` so editing a price automatically recomputes the row total and the grand total.

Step 5.1: Invoice models

models/invoiceitem.h

```
#ifndef INVOICEITEM_H
#define INVOICEITEM_H

#include <QString>

struct InvoiceItem {
    int    id          = -1;
    QString name;
    int    quantity    = 1;
    double price       = 0.0;
    double gstPercent  = 0.0;
    double cgst        = 0.0;
    double sgst        = 0.0;
    double igst        = 0.0;
    double total       = 0.0;
};

#endif
```

models/invoice.h

```
#ifndef INVOICE_H
#define INVOICE_H

#include <QString>
#include <QList>
#include "invoiceitem.h"

struct Invoice {
    int    id          = -1;
    QString invoiceNo;
    int    vendorId    = -1;
    QString invoiceDate;
    QString dueDate;
    double subtotal    = 0.0;
    double gstTotal    = 0.0;
    double grandTotal  = 0.0;
    QString status     = "Pending";
    QString notes;
    QList<InvoiceItem> items;
};

#endif
```

Step 5.2: Custom InvoiceItemModel

Why a custom model? `QStandardItemModel` stores strings in cells, which means parsing back and forth between numbers and text. A `QAbstractTableModel` holds typed C++ objects, so quantity is an int, price is a double, and your calculations stay clean. It is also how Qt expects you to handle real data.

models/invoiceitemmodel.h

```
#ifndef INVOICEITEMMODEL_H
#define INVOICEITEMMODEL_H

#include <QAbstractTableModel>
#include <QList>
#include "invoiceitem.h"

class InvoiceItemModel : public QAbstractTableModel {
    Q_OBJECT
public:
    enum Column { Name, Qty, Price, GstPct, CGST, SGST, IGST, Total, ColCount };

    explicit InvoiceItemModel(QObject* parent = nullptr);

    int rowCount(const QModelIndex& = QModelIndex()) const override;
    int columnCount(const QModelIndex& = QModelIndex()) const override;
    QVariant data(const QModelIndex& idx, int role) const override;
    bool setData(const QModelIndex& idx, const QVariant& v, int role) override;
    QVariant headerData(int section, Qt::Orientation o, int role) const override;
    Qt::ItemFlags flags(const QModelIndex& idx) const override;

    void addRow();
    void removeRow(int row);
    const QList<InvoiceItem>& items() const { return m_rows; }
    void setItems(const QList<InvoiceItem>& list);

    void setIntraState(bool sameState); // true => CGST+SGST, false => IGST

signals:
    void totalsChanged();

private:
    void recompute(int row);

    QList<InvoiceItem> m_rows;
    bool m_intraState = true;
};

#endif
```


models/invoiceitemmodel.cpp

```
#include "invoiceitemmodel.h"

InvoiceItemModel::InvoiceItemModel(QObject* p) : QAbstractTableModel(p) {}

int InvoiceItemModel::rowCount(const QModelIndex&) const { return m_rows.size(); }
int InvoiceItemModel::columnCount(const QModelIndex&) const { return ColCount; }

QVariant InvoiceItemModel::headerData(int s, Qt::Orientation o, int role) const {
    if (role != Qt::DisplayRole || o != Qt::Horizontal) return QVariant();
    static const QStringList h = {"Item", "Qty", "Price", "GST %",
                                   "CGST", "SGST", "IGST", "Total"};
    return h.value(s);
}

Qt::ItemFlags InvoiceItemModel::flags(const QModelIndex& idx) const {
    auto base = QAbstractTableModel::flags(idx);
    if (idx.column() <= GstPct) base |= Qt::ItemIsEditable;
    return base;
}

QVariant InvoiceItemModel::data(const QModelIndex& idx, int role) const {
    if (!idx.isValid()) return QVariant();
    if (role != Qt::DisplayRole && role != Qt::EditRole) return QVariant();

    const InvoiceItem& r = m_rows[idx.row()];
    switch (idx.column()) {
        case Name:    return r.name;
        case Qty:     return r.quantity;
        case Price:   return r.price;
        case GstPct:  return r.gstPercent;
        case CGST:    return QString::number(r.cgst, 'f', 2);
        case SGST:    return QString::number(r.sgst, 'f', 2);
        case IGST:    return QString::number(r.igst, 'f', 2);
        case Total:   return QString::number(r.total, 'f', 2);
    }
    return QVariant();
}

bool InvoiceItemModel::setData(const QModelIndex& idx, const QVariant& v, int role) {
    if (role != Qt::EditRole) return false;
    InvoiceItem& r = m_rows[idx.row()];
    switch (idx.column()) {
        case Name:    r.name = v.toString(); break;
        case Qty:     r.quantity = v.toInt(); break;
        case Price:   r.price = v.toDouble(); break;
        case GstPct:  r.gstPercent = v.toDouble(); break;
        default:      return false;
    }
    recompute(idx.row());
    emit dataChanged(index(idx.row(), 0), index(idx.row(), ColCount - 1));
    emit totalsChanged();
    return true;
}

void InvoiceItemModel::recompute(int row) {
    InvoiceItem& r = m_rows[row];
    double subtotal = r.quantity * r.price;
    double tax      = subtotal * r.gstPercent / 100.0;
    if (m_intraState) {
        r.cgst = tax / 2.0;
        r.sgst = tax / 2.0;
        r.igst = 0.0;
    } else {
```

```

        r.cgst = 0.0;
        r.sgst = 0.0;
        r.igst = tax;
    }
    r.total = subtotal + tax;
}

void InvoiceItemModel::addRow() {
    beginInsertRows(QModelIndex(), m_rows.size(), m_rows.size());
    m_rows.append(InvoiceItem());
    endInsertRows();
    emit totalsChanged();
}

void InvoiceItemModel::removeRow(int row) {
    if (row < 0 || row >= m_rows.size()) return;
    beginRemoveRows(QModelIndex(), row, row);
    m_rows.removeAt(row);
    endRemoveRows();
    emit totalsChanged();
}

void InvoiceItemModel::setItems(const QList<InvoiceItem>& list) {
    beginResetModel();
    m_rows = list;
    endResetModel();
    emit totalsChanged();
}

void InvoiceItemModel::setIntraState(bool same) {
    m_intraState = same;
    for (int i = 0; i < m_rows.size(); ++i) recompute(i);
    emit dataChanged(index(0,0), index(rowCount()-1, ColCount()-1));
    emit totalsChanged();
}

```

Note: When you call `beginInsertRows` / `endInsertRows` or `beginResetModel` / `endResetModel`, you are telling every view connected to this model to update. Skip these calls and the table will not refresh. Emit `dataChanged` after any cell value changes too.

Step 5.3: Wire the model into InvoiceWindow

```

// Inside InvoiceWindow constructor:
m_itemModel = new InvoiceItemModel(this);
m_itemTable = new QTableView;
m_itemTable->setModel(m_itemModel);
m_itemTable->horizontalHeader()->setStretchLastSection(true);
m_itemTable->setEditTriggers(QAbstractItemView::DoubleClicked
    | QAbstractItemView::SelectedClicked);

connect(m_itemModel, &InvoiceItemModel::totalsChanged,
        this, &InvoiceWindow::recalcTotals);

auto* addRowBtn = new QPushButton("+ Add item");
auto* delRowBtn = new QPushButton("- Remove item");
connect(addRowBtn, &QPushButton::clicked, m_itemModel, &InvoiceItemModel::addRow);
connect(delRowBtn, &QPushButton::clicked, this, [this]{
    auto sel = m_itemTable->selectionModel()->selectedRows();
    if (!sel.isEmpty()) m_itemModel->removeRow(sel.first().row());
});

```

Checkpoint for Phase 5

You can add rows to the invoice item table, type a quantity, price, and GST percent, and watch the CGST, SGST, and Total columns update in real time. Toggle between intra state (CGST+SGST) and inter state (IGST) and confirm the split changes.

Phase 6: GST Calculation Engine

Phase 5 already did per row math inside the model. This phase pulls calculation rules into a dedicated GstCalculator class so the logic is testable and reusable. The model calls it instead of doing math inline.

services/gstcalculator.h

```
#ifndef GSTCALCULATOR_H
#define GSTCALCULATOR_H

#include "models/invoiceitem.h"
#include "models/invoice.h"

class GstCalculator {
public:
    enum Mode { Intra, Inter };

    // Recomputes per row tax fields and total. Pure function.
    static void applyToItem(InvoiceItem& item, Mode mode);

    // Sums all rows and fills invoice level totals.
    static void applyToInvoice(Invoice& invoice, Mode mode);
};

#endif
```

services/gstcalculator.cpp

```
#include "gstcalculator.h"

void GstCalculator::applyToItem(InvoiceItem& it, Mode mode) {
    double subtotal = it.quantity * it.price;
    double tax      = subtotal * it.gstPercent / 100.0;

    if (mode == Intra) {
        it.cgst = tax / 2.0;
        it.sgst = tax / 2.0;
        it.igst = 0.0;
    } else {
        it.cgst = 0.0;
        it.sgst = 0.0;
        it.igst = tax;
    }
    it.total = subtotal + tax;
}

void GstCalculator::applyToInvoice(Invoice& inv, Mode mode) {
    double sub = 0.0, gst = 0.0;
    for (InvoiceItem& it : inv.items) {
        applyToItem(it, mode);
        sub += it.quantity * it.price;
        gst += it.cgst + it.sgst + it.igst;
    }
    inv.subtotal = sub;
    inv.gstTotal = gst;
    inv.grandTotal = sub + gst;
}
```

Refactor InvoiceItemModel::recompute to use this:

```
void InvoiceItemModel::recompute(int row) {
    GstCalculator::applyToItem(
```

```
        m_rows[row],  
        m_intraState ? GstCalculator::Intra : GstCalculator::Inter);  
    }
```

Worked example. Quantity 2, Price 1000, GST 18 percent, Intra state. Subtotal = $2 * 1000 = 2000$. Tax = $2000 * 0.18 = 360$. CGST = 180, SGST = 180, IGST = 0. Row total = 2360. If you change to Inter state, CGST and SGST become 0 and IGST becomes 360. Same row total.

Checkpoint for Phase 6

Calculations still work the same way after refactoring. The model file is shorter and the math now lives in one testable place.

Phase 7: Auto Fill Vendor with QCompleter

When the user types in the vendor field on the invoice form, suggestions should pop up. Picking one fills in the GST number, phone, and address automatically.

Step 7.1: Editable combo box with completer

```
#include <QComboBox>
#include <QCompleter>

void InvoiceWindow::setupVendorPicker() {
    m_vendorBox = new QComboBox;
    m_vendorBox->setEditable(true);
    m_vendorBox->setInsertPolicy(QComboBox::NoInsert);

    VendorService svc;
    for (const Vendor& v : svc.all()) {
        // Display name, store id in userData so we can look it up cheaply.
        m_vendorBox->addItem(v.name, v.id);
    }

    auto* completer = new QCompleter(m_vendorBox->model(), this);
    completer->setCaseSensitivity(Qt::CaseInsensitive);
    completer->setFilterMode(Qt::MatchContains);
    m_vendorBox->setCompleter(completer);

    connect(m_vendorBox, &QComboBox::currentIndexChanged,
            this, &InvoiceWindow::onVendorChosen);
}

void InvoiceWindow::onVendorChosen(int index) {
    if (index < 0) return;
    int vendorId = m_vendorBox->itemData(index).toInt();

    VendorService svc;
    Vendor v;
    if (svc.findById(vendorId, &v)) {
        m_gstNumber->setText(v.gst);
        m_vendorPhone->setText(v.phone);
        m_vendorAddress->setPlainText(v.address);

        // Decide GST mode based on whether vendor GST starts with the
        // same state code as your company GST.
        bool sameState = isSameState(v.gst, m_companyGst);
        m_itemModel->setIntraState(sameState);
    }
}
```

Note: Indian GST numbers start with a 2 digit state code. To detect intra vs inter state, compare the first two characters of the company GST and the customer GST. Defaulting to intra state when either is missing is fine for a portfolio project.

Checkpoint for Phase 7

Type two letters of any existing vendor name in the combo box. A dropdown shows matches. Pick one, the GST, phone, and address fields fill in automatically. If the vendor is in a different state, the GST table flips from CGST+SGST to IGST.

Phase 8: Invoice History with QSortFilterProxyModel

The history page lists every invoice with quick search and filtering. We use QSortFilterProxyModel to filter a source model without rewriting it from scratch.

Step 8.1: InvoiceService::listAll

```
QList<Invoice> InvoiceService::listAll() {
    QList<Invoice> list;
    QSqlQuery q(Database::instance().db());
    q.exec("SELECT i.id, i.invoice_no, v.name, i.invoice_date, "
          "        i.grand_total, i.status "
          "FROM invoices i "
          "LEFT JOIN vendors v ON v.id = i.vendor_id "
          "ORDER BY i.invoice_date DESC");
    while (q.next()) {
        Invoice inv;
        inv.id          = q.value(0).toInt();
        inv.invoiceNo    = q.value(1).toString();
        inv.invoiceDate  = q.value(3).toString();
        inv.grandTotal   = q.value(4).toDouble();
        inv.status       = q.value(5).toString();
        // store vendor name in notes temporarily for display
        inv.notes        = q.value(2).toString();
        list.append(inv);
    }
    return list;
}
```

Step 8.2: History view with proxy model

```
#include <QSortFilterProxyModel>
#include <QStandardItemModel>

HistoryWindow::HistoryWindow(QWidget* parent) : QWidget(parent) {
    m_source = new QStandardItemModel(this);
    m_proxy = new QSortFilterProxyModel(this);
    m_proxy->setSourceModel(m_source);
    m_proxy->setFilterKeyColumn(-1);           // search every column
    m_proxy->setFilterCaseSensitivity(Qt::CaseInsensitive);

    m_table = new QTableView;
    m_table->setModel(m_proxy);
    m_table->setSortingEnabled(true);

    m_search = new QLineEdit;
    m_search->setPlaceholderText("Search invoice no, vendor, status");
    connect(m_search, &QLineEdit::textChanged,
            m_proxy, &QSortFilterProxyModel::setFilterFixedString);

    auto* v = new QVBoxLayout(this);
    v->addWidget(m_search);
    v->addWidget(m_table);
    reload();
}

void HistoryWindow::reload() {
    m_source->clear();
    m_source->setHorizontalHeaderLabels(
        {"Invoice #", "Vendor", "Date", "Total", "Status"});

    InvoiceService svc;
    for (const Invoice& inv : svc.listAll()) {
        QList<QStandardItem*> row;
        row << new QStandardItem(inv.invoiceNo)
              << new QStandardItem(inv.notes)           // vendor name
              << new QStandardItem(inv.invoiceDate)
              << new QStandardItem(QString::number(inv.grandTotal, 'f', 2))
              << new QStandardItem(inv.status);
        row[0]->setData(inv.id, Qt::UserRole);
        for (auto* it : row) it->setEditable(false);
        m_source->appendRow(row);
    }
}
```

Checkpoint for Phase 8

History shows all invoices, newest first. Typing in the search box filters live. Clicking column headers sorts. Double clicking a row should open that invoice (wire this up to InvoiceWindow with a loadInvoice(id) method that fills all fields from the database).

Phase 9: PDF Export with QPainter and QPdfWriter

Qt does not have a Word style document API for PDF. You draw the invoice with QPainter, the same way you would paint on a widget, but the surface is a PDF page. This gives you total control over layout but you must do positioning yourself.

Step 9.1: PdfExporter skeleton

services/pdfexporter.h

```
#ifndef PDFEXPORTER_H
#define PDFEXPORTER_H

#include <QString>
#include "models/invoice.h"
#include "models/vendor.h"
#include "models/companysettings.h"

class PdfExporter {
public:
    bool exportInvoice(const Invoice& inv,
                      const Vendor& vendor,
                      const CompanySettings& company,
                      const QString& outPath,
                      QString* err);
};

#endif
```

services/pdfexporter.cpp

```
#include "pdfexporter.h"
#include <QPdfWriter>
#include <QPainter>
#include <QPageSize>
#include <QFile>

bool PdfExporter::exportInvoice(const Invoice& inv, const Vendor& v,
                                const CompanySettings& c,
                                const QString& outPath, QString* err) {
    QPdfWriter pdf(outPath);
    pdf.setPageSize(QPageSize(QPageSize::A4));
    pdf.setResolution(300);
    pdf.setPageMargins(QMarginsF(20, 20, 20, 20));

    QPainter p;
    if (!p.begin(&pdf)) { *err = "Cannot open PDF for writing"; return false; }

    const int W = pdf.width();
    int y = 0;

    // ----- Header -----
    QFont titleFont = p.font(); titleFont.setPointSize(22); titleFont.setBold(true);
    p.setFont(titleFont);
    p.drawText(0, y, W, 80, Qt::AlignLeft, c.companyName);
    p.drawText(0, y, W, 80, Qt::AlignRight, "TAX INVOICE");
    y += 90;

    QFont small = p.font(); small.setPointSize(10); small.setBold(false);
    p.setFont(small);
    p.drawText(0, y, W/2, 30, Qt::AlignLeft, c.address);
    p.drawText(W/2, y, W/2, 30, Qt::AlignRight,
               QString("Invoice #: %1").arg(inv.invoiceNo));
    y += 30;
    p.drawText(0, y, W/2, 30, Qt::AlignLeft,
               QString("GSTIN: %1").arg(c.companyGst));
    p.drawText(W/2, y, W/2, 30, Qt::AlignRight,
               QString("Date: %1").arg(inv.invoiceDate));
    y += 50;

    // ----- Bill to -----
    QFont bold = small; bold.setBold(true);
    p.setFont(bold);
    p.drawText(0, y, W, 30, Qt::AlignLeft, "Bill To:");
    p.setFont(small);
    y += 30;
    p.drawText(0, y, W, 30, Qt::AlignLeft, v.name); y += 25;
    p.drawText(0, y, W, 30, Qt::AlignLeft, v.address); y += 25;
    p.drawText(0, y, W, 30, Qt::AlignLeft,
               QString("GSTIN: %1").arg(v.gst)); y += 40;

    // ----- Item table header -----
    const int colX[] = {0, W*45/100, W*55/100, W*65/100, W*80/100, W};
    const int rowH = 35;

    p.setFont(bold);
    p.fillRect(QRect(0, y, W, rowH), QColor("#0b3d91"));
    p.setPen(Qt::white);
    p.drawText(colX[0]+8, y, colX[1]-colX[0], rowH, Qt::AlignVCenter|Qt::AlignLeft, "Item");
    p.drawText(colX[1], y, colX[2]-colX[1], rowH, Qt::AlignVCenter|Qt::AlignRight, "Qty");
    p.drawText(colX[2], y, colX[3]-colX[2], rowH, Qt::AlignVCenter|Qt::AlignRight, "Price");
    p.drawText(colX[3], y, colX[4]-colX[3], rowH, Qt::AlignVCenter|Qt::AlignRight, "GST");
    p.drawText(colX[4], y, colX[5]-colX[4]-8, rowH, Qt::AlignVCenter|Qt::AlignRight, "Total");
    p.setPen(Qt::black);
```

```

y += rowH;

// ----- Item rows -----
p.setFont(small);
for (const InvoiceItem& it : inv.items) {
    p.drawText(colX[0]+8, y, colX[1]-colX[0], rowH, Qt::AlignVCenter|Qt::AlignLeft, it.name);
    p.drawText(colX[1], y, colX[2]-colX[1], rowH, Qt::AlignVCenter|Qt::AlignRight, QString::number(it.qty));
    p.drawText(colX[2], y, colX[3]-colX[2], rowH, Qt::AlignVCenter|Qt::AlignRight, QString::number(it.price));
    p.drawText(colX[3], y, colX[4]-colX[3], rowH, Qt::AlignVCenter|Qt::AlignRight, QString::number(it.total));
    p.drawText(colX[4], y, colX[5]-colX[4]-8, rowH, Qt::AlignVCenter|Qt::AlignRight, QString::number(it.total));
    p.drawLine(0, y+rowH, W, y+rowH);
    y += rowH;
}

// ----- Totals -----
y += 20;
p.setFont(bold);
auto totalRow = [&](const QString& label, double amount){
    p.drawText(W*60/100, y, W*30/100, rowH, Qt::AlignRight|Qt::AlignVCenter, label);
    p.drawText(W*90/100, y, W*10/100, rowH, Qt::AlignRight|Qt::AlignVCenter,
        QString::number(amount, 'f', 2));
    y += rowH;
};
totalRow("Subtotal:", inv.subtotal);
totalRow("GST Total:", inv.gstTotal);
totalRow("Grand Total:", inv.grandTotal);

// ----- Footer -----
y += 80;
p.setFont(small);
p.drawText(0, y, W, 40, Qt::AlignLeft,
    "Notes: " + inv.notes);
p.drawText(0, pdf.height()-60, W, 40, Qt::AlignCenter,
    "Thank you for your business.");

p.end();
return true;
}

```

Step 9.2: Trigger export from the UI

```

void InvoiceWindow::onExportPdfClicked() {
    QString file = QFileDialog::getSaveFileName(
        this, "Export PDF",
        QString("invoice_%1.pdf").arg(m_invoiceNoEdit->text()),
        "PDF Files (*.pdf)");
    if (file.isEmpty()) return;

    Invoice inv = collectInvoiceFromUI();
    Vendor v; VendorService().findById(inv.vendorId, &v);
    CompanySettings c = SettingsService().load();

    PdfExporter exp;
    QString err;
    if (!exp.exportInvoice(inv, v, c, file, &err)) {
        QMessageBox::warning(this, "Export failed", err);
    } else {
        QMessageBox::information(this, "Done", "Saved to " + file);
    }
}

```

Checkpoint for Phase 9

Click Export PDF. A file dialog opens. After saving, open the PDF. The header should show your company name, the bill to block shows the customer, items table is aligned, and totals line up on the right. If text overlaps, increase the y spacing or shrink the font.

Phase 10: Settings, Persistence, and Dark Mode

QSettings stores key value pairs in the platform's native location (registry on Windows, plist on macOS, ini file on Linux). Use it for user preferences. Use the database for application data.

services/settingservice.cpp (snippet)

```
#include <QSettings>

void SettingsService::saveWindowState(const QByteArray& geom,
                                      const QByteArray& state) {
    QSettings s;
    s.setValue("ui/geometry", geom);
    s.setValue("ui/state", state);
}

void SettingsService::restoreWindowState(QMainWindow* w) {
    QSettings s;
    w->restoreGeometry(s.value("ui/geometry").toByteArray());
    w->restoreState(s.value("ui/state").toByteArray());
}

void SettingsService::setTheme(const QString& name) {
    QSettings s;
    s.setValue("ui/theme", name);
}

QString SettingsService::theme() {
    return QSettings().value("ui/theme", "light").toString();
}
```

Set the QApplication organization and application name once in main.cpp so QSettings uses a stable key:

```
QApplication::setOrganizationName("YourName");
QApplication::setApplicationName("InvoiceApp");
```

Dark mode with QSS

Qt Style Sheets (QSS) use CSS-like syntax. Put two files in resources/styles/: light.qss and dark.qss. Load the chosen one at startup with QFile and qApp->setStyleSheet.

```
void applyTheme(const QString& name) {
    QFile f(QString(":/styles/%1.qss").arg(name));
    if (f.open(QIODevice::ReadOnly | QIODevice::Text))
        qApp->setStyleSheet(QString::fromUtf8(f.readAll()));
}
```

A starter dark.qss:

```
QWidget { background:#1e1e1e; color:#e4e4e4; }
QPushButton { background:#3a3a3a; border:1px solid #555;
padding:6px 12px; border-radius:4px; }
QPushButton:hover { background:#4a4a4a; }
QLineEdit, QTextEdit, QComboBox {
background:#2a2a2a; border:1px solid #555; padding:4px;
}
QHeaderView::section { background:#0b3d91; color:white; padding:6px; }
QTableView { gridline-color:#444; selection-background-color:#1a4fb0; }
```

Checkpoint for Phase 10

Resize the main window, close the app, reopen, the window has the same size and position. Toggle theme in settings, the whole app switches between light and dark immediately.

Phase 11: Optimization, Refactoring, Packaging

Refactoring checklist

- Move all magic strings (status names, table names) into constants in a single header.
- Wrap any block that runs more than one INSERT or UPDATE in a transaction (QSqlDatabase::transaction / commit). For invoices with many items, this is a 10x speedup and prevents half written rows.
- Replace QStandardItemModel with custom models where you find yourself parsing string cells back to numbers.
- Add indexes for columns you search by often. Example: CREATE INDEX idx_invoices_date ON invoices(invoice_date).
- Remove dead code, unused includes, and commented out blocks before you call it done.

Validators on all numeric input

```
#include <QDoubleValidator>
#include <QRegularExpressionValidator>

auto* priceEdit = new QLineEdit;
priceEdit->setValidator(new QDoubleValidator(0.0, 9999999.0, 2, this));

auto* gstEdit = new QLineEdit;
QRegularExpression gstRx("[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$");
gstEdit->setValidator(new QRegularExpressionValidator(gstRx, this));
```

Packaging the app

Windows: run windeployqt from the terminal after building:

```
windeployqt --release --no-translations --no-system-d3d-compiler ^
--no-opengl-sw path\to\InvoiceApp.exe
```

This copies every required DLL into the build folder. Zip and ship.

Linux: use linuxdeployqt or build an AppImage.

macOS: use macdeployqt path/to/InvoiceApp.app

Final checkpoint

Fresh clone, fresh build, run from a folder with no existing database. The first run should create the database, prompt for signup, and walk through every feature without a crash. If you can do this on a friend's machine, you are done.

Appendix A: Common Qt Mistakes and How to Debug Them

Symptom	Fix
Undefined reference to vtable for X	You added Q_OBJECT to a class but forgot to add the .cpp file to CMakeLists.txt, so MOC
Signal/slot connection silently fails	Old style SIGNAL/SLOT macros do not check at compile time. Use the new pointer to m
QString returns nothing but no error	You called q.exec() but forgot q.next() before reading. Or your prepared statement bind v
UI does not update when model changes	You modified the underlying data without emitting dataChanged or wrapping in beginInse
Database is locked	SQLite allows only one writer at a time. If you opened two connections (often by accident
Memory leak from missing parent	If you create a widget with new but do not give it a parent, Qt cannot delete it for you. Alw
App works in debug build, crashes in release	Always uninitialized member variables. Initialize every member in the header (int
Translations or special characters look broken	Use QString and toUtf8 / fromUtf8 at file and DB boundaries. Avoid char* and std::string

Appendix B: Final Project Checklist

Tick each item before you call the project done.

- 1 All 5 tables present in the database and foreign keys enabled
- 2 Signup creates a row with unique salt per user
- 3 Login fails on wrong password, succeeds on correct
- 4 Dashboard shows live stats (total invoices, total revenue, pending count)
- 5 Vendor CRUD works and search filters in real time
- 6 Invoice form: add and remove rows, totals update per keystroke
- 7 Intra vs inter state correctly splits between CGST+SGST and IGST
- 8 Auto fill: typing vendor name fills GST, phone, address
- 9 Saved invoice can be reopened from history with all fields intact
- 10 PDF export produces a properly formatted A4 invoice with company header, bill to, item table, and totals
- 11 Window size and theme persist across restarts
- 12 App runs from a fresh folder with no existing database without crashing
- 13 Code is split across UI / Service / Model / Core folders with no SQL inside UI files
- 14 CMakeLists includes every .cpp and .h file
- 15 windeployqt (or platform equivalent) produces a runnable folder you can hand to someone else

Where to go next

Once the base app is solid, the future improvements section of your plan (dark mode polish, CSV/Excel export, charts, email, cloud sync) are all add ons that do not require redesigning anything. Each one is a self contained service you bolt on. Build the base first, ship it, then extend.